SESIÓN 1 · SEGUNDA MITAD

Clase 14 · CI/CD y promoción entre ambientes

A producción se llega por un camino, no por un clic

Docente: Manuela Larrea Gómez

iData
Global Data & AI

CI, entrega continua y despliegue continuo: tres preguntas distintas

Muchos equipos dicen CI/CD y solo tienen la primera

Término	Pregunta que responde	En nuestro ci.yml
Integración continua (CI)	¿Este cambio rompe algo?	test (ruff + pytest unit, incluidas las pruebas del bundle y del pipeline) · integration (Spark local) · validate (el bundle contra el workspace real)
Entrega continua (CD)	¿Siempre tengo una versión lista para producción?	deploy-dev en cada merge a main · deploy-qa en cada tag v*
Despliegue continuo	¿Cada cambio llega solo a producción?	No, a propósito: deploy-prod espera una aprobación humana

¿Por qué no despliegue continuo?

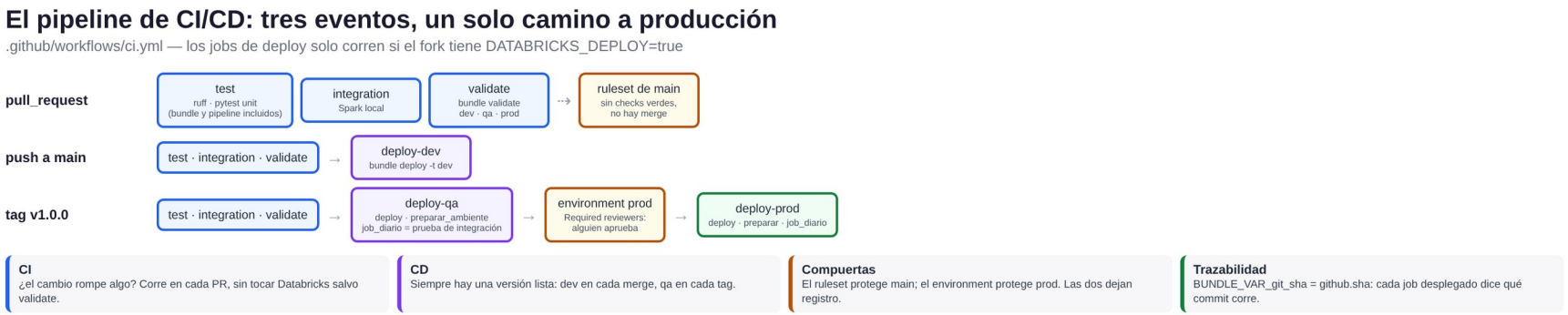
El pronóstico lo consumen planeación y terceros. La aprobación deja escrito quién decidió publicar qué versión.

La decisión no desaparece, cambia de dueño

Equipos maduros la automatizan con criterios medibles. Para el modelo, eso es la clase 15: decidir_promocion().

El pipeline: tres eventos, un solo camino a producción

.github/workflows/ci.yml · los jobs de deploy solo corren si el fork tiene DATABRICKS_DEPLOY = true



pull_request → CI · push a main → CI + deploy-dev · tag v1.0.0 → CI + deploy-qa → [aprobación] → deploy-prod

concurrency por ref

Dos promociones del mismo tag no corren a la vez: dos deploys al mismo ambiente se pisan.

permissions: contents: read

Mínimo privilegio para el token de GitHub del pipeline.

El pipeline se prueba a sí mismo

tests/unit/test_cicd.py lee ci.yml y exige sus reglas: needs, environment, sin tokens.

Con qué identidad despliega el pipeline

La regla de fondo: un secreto que no existe no se puede filtrar

Mecanismo	Qué guarda GitHub	Riesgo	Dónde
Token personal (PAT)	DATABRICKS_TOKEN: un secreto que vence	se filtra, se olvida renovar, despliega como una persona	hoy: Free Edition, un estudiante, su fork
Service principal con OAuth	client id + secreto, o un token generado	secreto de larga vida que hay que rotar	equipos sin federación
Federación de identidad (OIDC)	nada secreto: GitHub emite un token por corrida	la política define qué repo y qué environment pueden desplegar	lo recomendado en producción

OIDC en GitHub Actions

```
permissions: { id-token: write, contents: read }
env:
  DATABRICKS_AUTH_TYPE: github-oidc
  DATABRICKS_HOST:      ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: <service principal>
```

En XM, sobre Azure

Azure DevOps: service connection con workload identity federation hacia un service principal de Entra ID por ambiente. La política ata el permiso de desplegar a prod a un repositorio y un environment, no a una persona.

Dos compuertas: el ruleset protege main; el environment protege prod

Sin ellas, el CI es una sugerencia y la aprobación es una costumbre

Ruleset sobre main

Require a pull request before merging · Require status checks to pass: test e integration.

Efecto: un PR en rojo no se puede fusionar, y git push origin main se rechaza. Eso es "un merge que se salta CI": no pasa.

Environment prod con Required reviewers

deploy-prod declara environment: prod y needs: deploy-qa. GitHub lo deja en Waiting hasta que un revisor aprueba, y guarda quién y cuándo.

Ojo: en planes gratuitos, los required reviewers solo existen en repositorios públicos. Por eso el fork es público.

needs y environment protegen cosas distintas. needs: prod no corre si qa falló. environment: aun con qa en verde, prod espera a una persona. Quitar needs hace que qa y prod corran en paralelo: el tag llega a prod antes de saber si qa funcionó.

qa se prueba corriendo el job; cada job dice qué commit corre

El chequeo de frescura de la clase 12 es la prueba de integración de la promoción

.github/workflows/ci.yml

```yaml
deploy-qa:
  - databricks bundle deploy -t qa
  - databricks bundle run preparar_ambiente -t qa
  - databricks bundle run job_diario -t qa
    # si el chequeo falla, el comando falla,
    # y el tag no llega a prod
```

trazabilidad

```yaml
env:
  BUNDLE_VAR_git_sha: ${{ github.sha }}

# resources/job_diario.yml
tags:
  git_sha: ${var.git_sha}
```

Pregunta del auditor	Dónde se responde
¿Qué versión corre en cada ambiente?	tag git_sha de cada job desplegado (dev, qa, prod)
¿Con qué identidad?	run_as del job · creador del despliegue
¿Quién aprobó y cuándo?	GitHub → environment prod → historial de despliegues · en Azure: Azure DevOps y system.access.audit

Promover el código, promover el modelo

El código siempre viaja por Git y el pipeline. La pregunta es qué hacemos con el modelo

Promover el modelo o promover el código

El código siempre viaja por Git y el pipeline. La pregunta es qué hacemos con el modelo.

Promover el modelo (clases 13 y 14)



- qa prueba **exactamente** los pesos que irán a prod.
- Rápido y barato: se entrena una sola vez.
- Riesgo: prod sirve un modelo entrenado con datos de dev.
- El tag `origen` guarda el linaje.

Promover el código y reentrenar (clase 15)



- Lo recomendado por Databricks: *deploy code, not models*.
- Los datos de prod no salen de prod, ni dentro de un modelo.
- Cada ambiente compara su challenger con su champion antes de promover.
- Riesgo: el modelo de prod no es idéntico al probado en qa: la regla de promoción tiene que ser explícita.

En XM, con un workspace de prod que lee la fuente oficial, el camino de la derecha es el destino; el de la izquierda sirve para arrancar un ambiente nuevo.

Databricks recomienda por defecto deploy code, not models: los datos de producción no deberían salir de producción, ni dentro de un modelo.

Lo que hacemos hoy, y cuándo está bien

copy_model_version con tag origen. Sirve para arrancar un ambiente nuevo o cuando entrenar en el ambiente es caro o no está permitido.

A dónde vamos (clase 15)

El ambiente entrena su candidato con sus datos, lo compara con su champion en días que ninguno vio, y promueve solo si gana.

GitHub Actions y Azure DevOps, lado a lado

.github/workflows/ci.yml · ci/azure-pipelines.yml + ci/plantilla-bundle.yml (referencia)

Concepto	GitHub Actions	Azure DevOps
Unidad	workflow → jobs → steps	pipeline → stages → jobs → steps
Dependencias	needs: deploy-qa	dependsOn: DeployQA
Aprobación	environment prod + Required reviewers	Environments → prod → Approvals and checks
Secretos e identidad	Secrets · OIDC (github-oidc)	Variable groups · service connection con federación (Entra ID)
Reutilización	acciones y workflows reutilizables	plantillas: ci/plantilla-bundle.yml
Instalar la CLI	databricks/setup-cli	script de install.sh de setup-cli
Commit desplegado	BUNDLE_VAR_git_sha: ${{ github.sha }}	BUNDLE_VAR_git_sha: $(Build.SourceVersion)

La herramienta cambia; el diseño no: CI en cada PR, qa antes de prod, aprobación con registro y la identidad del pipeline, no la de una persona.

Lab 14: de un tag a producción, con pruebas y una aprobación

clases/c14-cicd/notebooks/lab14_cicd_promocion · fork público · Actions habilitado

1

Configurar el fork

Token · 2 secrets · DATABRICKS_DEPLOY · environments · ruleset de main

2

TODO · deploy-prod

needs: deploy-qa · environment: prod · correr preparar_ambiente y job_diario en prod

3

a · b · c

PR que rompe un test (rojo, merge bloqueado) → arreglar → merge → deploy-dev

4

d

git push directo a main: rechazado por el ruleset

5

e

tag v1.0.0 → deploy-qa → receso → Approve → deploy-prod → auditoría

Criterios de aceptación

- test_cicd.py sin skip
- Un PR detenido y un push rechazado
- v1.0.0 en qa y prod, con aprobación
- qa y prod con el mismo git_sha
- Dos preguntas · verificar() en OK

Si qa falla en el chequeo

No es un error del pipeline: es la prueba funcionando.
Revisa si tu landing tiene el archivo tardío del lab 12.

Duración: 45 min + 20 min tras el receso

Cierre de la sesión 1: lo que queda operando, y la sesión 2

Tags s13 y s14 en el repositorio de la docente: git diff s13 / s14 para compararte

Hoy (Free Edition)	En XM (Azure Databricks)
Ambientes = esquemas	Workspaces y catálogos por ambiente, con binding
run_as: tu usuario	Service principal por ambiente
Token personal en GitHub	OIDC o service connection con federación
Aprobación: tú mismo	Grupo de aprobadores, separación de funciones
Auditoría: GitHub + tags	+ system.access.audit

Sesión 2: el modelo se cuida solo

Monitoreo de error y drift, política de reentrenamiento como código, promoción solo si el candidato gana, alerta SQL, runbook y cierre del curso.

Traer: labs 13 y 14 con verificar() en OK y el PR fusionado.

Gracias

DOCENTE · Manuela Larrea Gómez

Lo que se llevan hoy: el mismo código en tres ambientes, un pipeline que se prueba a sí mismo, y un camino a producción que deja rastro.